

Design Report

Authors: Yigit Sarp Avci (2023400048) & Mehmet Dogukan Sungu (2023400210)

1 Scoping Decision

For this project, I implemented **both Static (Lexical) and Dynamic Scoping** as a bonus feature (B4). By default, the interpreter uses static scoping.

- **Static Scoping:** Variables are resolved in the environment where the function was defined. This is the standard in most modern languages (Python, Java, C). It makes code easier to reason about because the behavior of a function depends only on its definition context, not where it is called.

- **Dynamic Scoping:** Variables are resolved in the environment where the function is called. While rarer, it is found in languages like Emacs Lisp. It allows for "implicit" parameters but can lead to fragile code.

Comparison Example:

```
let x = 10;  
let f = fun () -> x end;  
let g = fun () -> let x = 20; f() end;  
print(g());
```

- Under **Static Scoping**, `f()` captures `x=10` from the global scope. Output: ``10``.

- Under **Dynamic Scoping**, `f()` looks up `x` in the caller's frame (``g``), where `x=20`. Output: ``20``.

2 Evaluation Strategy

The interpreter implements **Call-by-Value** (strict evaluation). This means all arguments are fully evaluated to values **before** being bound to function parameters and executing the function body.

Difference with Call-by-Name:

Under Call-by-Name, arguments are passed as unevaluated expressions and re-evaluated every time the parameter is accessed.

Example of different behavior:

```
let loop = fun () -> loop() end;
```

```
let ignore = fun (x) -> 42 end;
print(ignore(loop()));
```

- **Call-by-Value:** The interpreter tries to evaluate `loop()` before calling `ignore`, leading to an infinite recursion (stack overflow).

- **Call-by-Name:** Since `ignore` never uses its parameter `x`, `loop()` is never evaluated.
Output: `42`.

3 Closure Representation

In this interpreter, a **Closure** is represented as a dedicated object/class containing three components:

1. **Parameters:** A list of formal parameter names.
2. **Body:** The AST block of the function.
3. **Defining Environment:** A reference to the `'Environment'` frame where the function was defined.

When the function is called (under static scoping), the new environment frame's parent is set to this captured "Defining Environment", ensuring lexical scoping.

4 Environment Management & Tracing Exercise

The environment is implemented as a linked list of frames (activation records). Each frame contains:

- A dictionary of bindings (`'name -> value'`).
- A `'parent'` pointer to the enclosing frame.

Tracing Exercise

Program:

```
let x = 1;
let f = fun (a) -> let y = a + x; fun () -> x = x + 1; y + x end end;
let g = f(10);
print(g());
print(g());
```

1. **Global Frame (F0):** `{x: 1, f: <Closure>}`. Parent: None.
2. **Call `f(10)`:** Creates **Frame F1**. Parent: **F0**.
 - Bindings: `{a: 10}`.
 - `'let y = a + x; -> y = 10 + 1 = 11'`.
 - Bindings: `{a: 10, y: 11}`.
 - Returns an inner closure capturing **F1**.
3. **Assignment `let g = ...`:** **F0** now is `{x: 1, f: ..., g: <Closure(captures F1)>}`.
4. **First Call `g()`:** Creates **Frame F2**. Parent: **F1** (captured env).
 - `'x = x + 1; -> Looks up 'x'. Not in F2, not in F1. Found in F0. Updates F0.x to '2'.`

- ``y + x`` -> ``y`` found in **F1** (``11``), ``x`` found in **F0** (``2``). Result: ``13``.

- **Output: 13.**

5. **Second Call ``g()``:** Creates **Frame F3**. Parent: **F1**.

- ``x = x + 1`;` -> Updates **F0.x** to ``3``.

- ``y + x`` -> ``y`` in **F1** (``11``), ``x`` in **F0** (``3``). Result: ``14``.

- **Output: 14.**

Logical Operators

The interpreter supports ``and``, ``or``, and ``not`` with standard precedence (``not`` > ``and`` > ``or``). Logical operators feature short-circuit evaluation, ensuring efficiency and preventing errors in guarded expressions (e.g., ``if x != 0 and 10 / x > 1``).

Why ``y`` does not change?

``y`` is bound in **F1** during the execution of ``f(10)``. Since **F1** is never modified (only **F0.x** is updated), ``y`` remains ``11`` for all calls to ``g``.

5 Challenges

- **Implementing Implicit Recursion:** To allow ``let f = fun ...`` to be recursive, the environment must contain ``f`` before the function body is evaluated. I handled this by extending the environment with the closure, and since the closure captures the environment by reference, it sees itself.

- **Handling Nested Comments:** Regular expression-based lexing cannot handle nested structures like ``(* (* *) *)``. I implemented a custom iterative state-machine within the Lexer to track comment depth manually, ensuring 100% compliance with complex commenting rules.

- **Operator Precedence:** Implementing the correct precedence for ``not`` and unary ``-`` vs binary operators required careful structuring of the recursive descent levels.

- **Semicolon Logic:** Balancing the "statement needs semicolon" vs "last expression doesn't" required a more flexible ``parse_block`` implementation.

6 Grammar Ambiguity

- **Dangling Else:** The language uses an ``end`` keyword to terminate ``if`` blocks, which inherently resolves the dangling else ambiguity. An ``else`` always belongs to the nearest ``if`` that hasn't been closed by an ``end``.

- **Operator Associativity:** I implemented left-associativity for most operators using ``while`` loops in the parser. For comparison operators, I made them non-associative as per the spec, meaning ``a == b == c`` is a syntax error.